

Tema:

Grid Layout

План

1. Grid Layout. Загальні відомості. Основні терміни.
2. Встановлення параметрів контейнера.
3. Розташування елементів в сітці
4. Шаблон на основі назв областей (grid-area)
5. Відступи
6. Вирівнювання елементів
7. Вирівнювання сітки всередині контейнера
8. Явна та неявна сітка
9. Комплексна властивість grid
10. Grid vs browsers DevTools

Grid Layout. Загальні відомості.

CSS Grid Layout - це **двовимірна система розміщення елементів** на веб-сторінці, яка дозволяє створювати макети веб-сторінок на основі сітки з горизонтальних та вертикальних ліній. Тобто дана система **усуває потребу** вдаватися у **складності позиціонування** або **вбудовування одних елементів в інші**, оскільки бере на себе всю важку роботу, а розробнику забезпечує повний контроль.

Принцип роботи **grid чимось схожий на таблиці**. За допомогою grid можна працювати з так званими grid-комірками, позиціонуючи елементи по вертикалі та горизонталі одночасно.

CSS завжди використовувався для створення макетів веб-сторінок, однак завжди складалося враження, що бракує інструментарію для створення макетів і потрібно використовувати різні LifeHack-и. Спочатку, на зорі верстки, для створення макетів використовували таблиці, потім плаваючі елементи, позиціонування та вбудовані блоки, але всі ці методи були, по суті, хаками та не мали багатьох важливих функцій (наприклад, вертикального центрування). Flexbox - чудовий інструмент компонування, однак має односпрямований потік, який забезпечує свою зону використання, в тому числі разом з Grid.

Grid — це перший CSS-модуль, **створений спеціально для вирішення проблем макетування** при створенні веб-сайту.

Grid. Основні терміни.

Grid-контейнер: батьківський елемент, до якого застосовується властивість `display: grid`;

Grid-елемент : дочірній елемент, **прямий** нащадок grid-контейнера (тобто елементи `.item` є елементами сітки, але елемент `.sub-item` **не є** такими). Підпорядковується сконфігурованим правилам розкладки grid-layout.

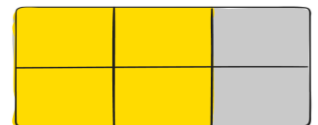
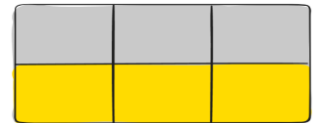
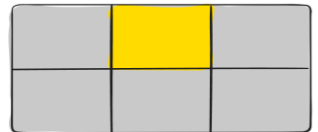
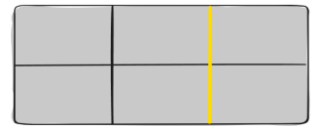
Grid-лінія : розділова лінія, що формує структуру grida. Може бути як вертикальною (grid-лінія стовбця), так і горизонтальною (grid-лінія рядка). Розташовується по обидва боки від колонки чи ряду. Використовується для прив'язки grid-елементів.

Grid-комірка: простір між сусідніми grid-лініями. Одиниця grid-сітки.

Grid-смуга, трек (тобто ряд чи колонка): простір між двома сусідніми grid-лініями (наприклад: смуга сітки **між лініями сітки другого та третього** рядків, чи смуга сітки **між лініями сітки другого та третього** стовбців).

Grid-область: область, обмежена чотирма grid-лініями. Може складатися з будь-якої кількості комірок як по горизонталі, так і по вертикалі.

```
<div class="container">
  <div class="item"> </div>
  <div class="item">
    <p class="sub-item"> </p>
  </div>
  <div class="item"> </div>
</div>
```



Властивість **display: grid**;

Якщо елементу встановлено властивість **display** зі значенням **grid**, то такий елемент стає grid-контейнером. Дочірні елементи цього контейнера починають підкорятися правилам grid-розкладки. Зовні grid-контейнер поводить себе як блок.

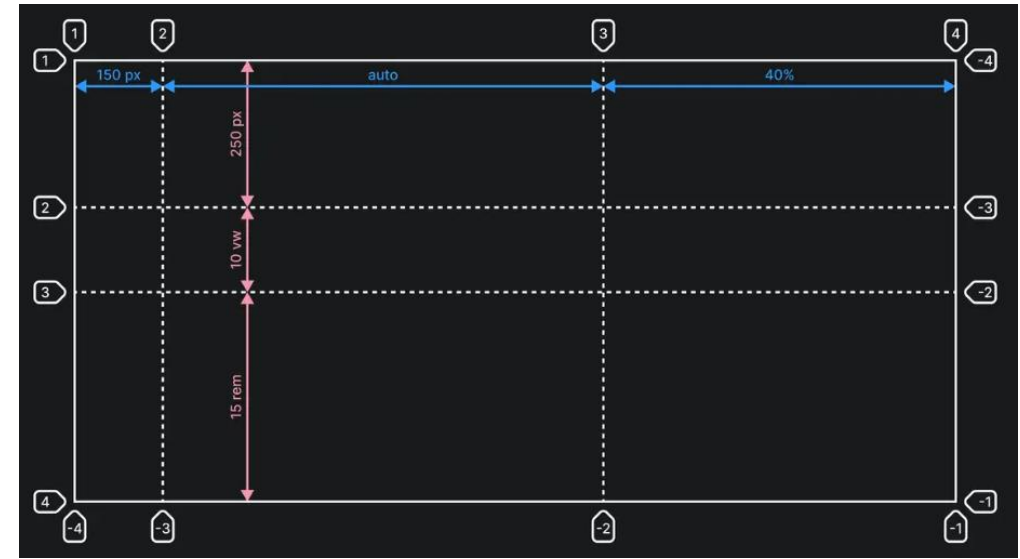
```
.container {  
    display: grid; /* or display: inline-grid; */  
}
```

Практично аналогічне попередньому, значення **inline-grid** - за тим винятком, що зовні grid-контейнер, створений за допомогою **inline-grid**, буде вести себе як рядковий елемент.

Властивості `grid-template-columns`, `grid-template-rows`

Властивості `grid-template-columns` та `grid-template-rows` задають розміри та кількість колонок чи рядів `grid`-розкладки

```
.container {  
  display: grid;  
  /* Буде створено 3 колонки */  
  grid-template-columns: 150px auto 40%;  
  /* Буде створено 3 ряди */  
  grid-template-rows: 250px 10vw 15rem;  
}
```



Можна явно називати `grid-лінії`, використовуючи для цього квадратні дужки:

```
.container {  
  display: grid;  
  grid-template-columns: [start] 250px [line2] 400px [line3] 600px [end];  
  grid-template-rows: [row1-start] 15rem [row1-end] 30vh [last];  
}
```

Властивості `grid-template-columns`, `grid-template-rows`

Деякі допустимі значення `grid-template-columns` та `grid-template-rows`:

`grid-template-columns`: `100px 100px 200px` - створює сітку з трьох колонок. Перша колонка дорівнює `100px`, друга – `100px`, третя – `200px`

`grid-template-columns`: `min-content max-content fit-content(10em)` - створює сітку з трьох колонок. Перша колонка має розмір `min-content`, друга – `max-content`, третя колонка має ширину `max-content`, якщо вміст не перевищує `10em`, якщо перевищує – обмежується шириною `10em`.

`grid-template-columns`: `1fr 1fr 1fr` - створює сітку з трьох колонок, використовуючи одиницю вимірювання `fr` (тобто `fraction`). Таким чином доступний простір `grid`-контейнера розподіляється між трьома колонками.

`grid-template-columns`: `repeat(2, 10em 1fr)` - створює сітку на чотири колонки з розмірами колонок `10em 1fr 10em 1fr` (тобто `2 * 10em 1fr`), так як використовується оператор `repeat()` з двома повтореннями шаблону колонок.

`grid-template-columns`: `repeat(auto-fill, 200px)` – заповнює контейнер такою кількістю колонок шириною по `200px`, скільки поміститься, залишаючи в кінці пропуск, якщо залишилося трохи додаткового місця.

`grid-template-columns`: `repeat(auto-fill, minmax(200px, 1fr))` – заповнює контейнер такою кількістю колонок шириною по `200px`, скільки поміститься, після чого розподіляє простір, що залишився, між цими створеними колонками.

`grid-template-columns`: `[full-start] 1fr [content-start] 3fr [content-end] 1fr [full-end]` - створює сітку з трьох колонок. Перша та третя колонки мають ширину, рівну 1 частині доступного простору (`1 fr`), а третя – 3 частинам (`3fr`). При цьому задаються **назви лініям** шляхом їх вказання у квадратних дужках.

Розміри треків (колонок та рядків)

Для створення параметрів колонок та рядків можна використовувати будь-які **одиниці виміру довжини** чи **відсотки**. Якщо сума ширин треків менша за доступну в grid-контейнері, за замовчуванням треки вибудовуються на початку контейнера, а в кінці залишається вільне місце.

Також можна використовувати **ключові слова** та **min-content**, **max-content** та **fit-content()**

Використання **min-content** генерує трек, який буде настільки маленьким, наскільки це можливо, щоб не викликати переповнення вмістом. Отже, при його використанні в якості розміру колонки, вміст м'яко переноситься в тих місцях, де це можливо. Трек отримує ширину, що дорівнює найдовшому слову в колонці або найбільшому елементу фіксованого розміру.

Використання **max-content** призведе до того, що вміст взагалі не переноситься, що може призвести до переповнення.

Ключове слово **fit-content** може бути використане тільки разом із параметром, що передається йому в дужках. Це значення стає максимумом, до якого може розтягуватись цей трек. Тобто трек може бути менше переданого значення, але ніколи не буде більше за нього.

Ширину треків можна задавати за допомогою одиниць вимірювання **fr** (від слова **fraction** - частка). Це **гнучка одиниця виміру**, яка **розраховується виходячи з величини вільного простору у grid-контейнері**. За умовчанням "доступний простір" - це **не весь** простір контейнера. Якщо якась із колонок містить елемент фіксованого розміру або довге слово, яке не може бути перенесене на новий рядок, простір під цю колонку буде виділено до того місця, доки не стане можливим перенесення. Тобто вільне місце розраховується після того, як місце віддано всім фіксованим розмірам.

Розташування елемента в сітці

Властивості **grid-column-start**, **grid-column-end**, **grid-row-start** та **grid-row-end** визначають розташування елемента в сітці за допомогою певних ліній сітки.

grid-column-start / **grid-row-start** – це колонка/рядок, де починається елемент
grid-column-end / **grid-row-end** – це колонка/рядок, де елемент закінчується.

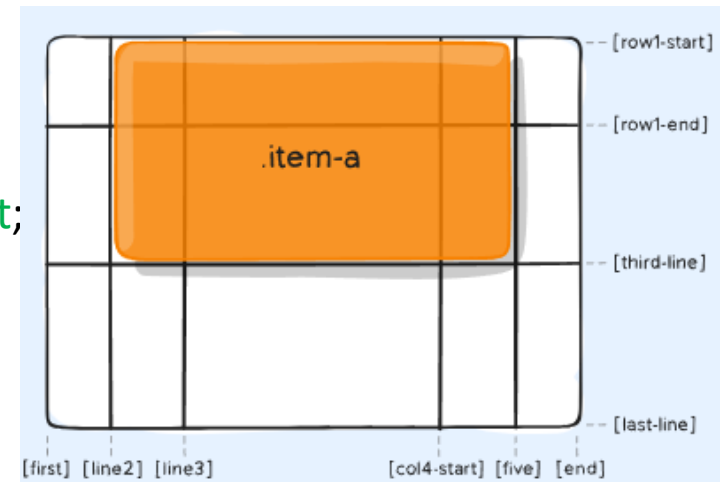
Властивості можуть приймати значення:
<line> – число (номер), або назва (ім'я) лінії сітки

span <number> – елемент охоплюватиме вказану кількість (рядків/стовбців) сітки

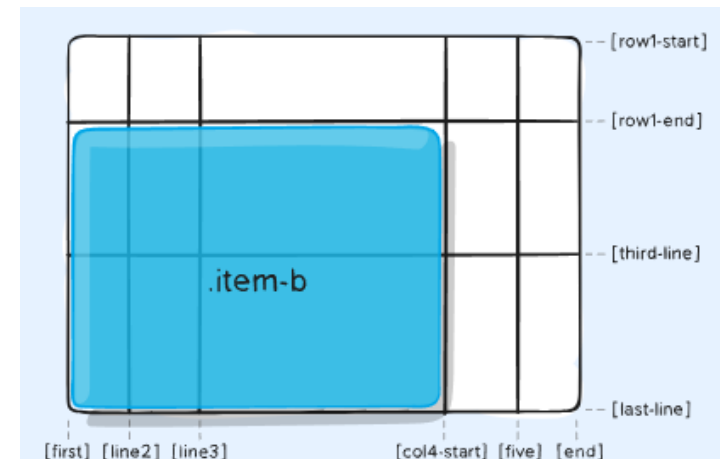
span <name> – елемент охоплюватиметься простір до рядка (стовбця) із вказаною назвою

auto – вказує на автоматичне розміщення, або діапазон за замовчуванням

```
.item-a {  
  grid-column-start: 2;  
  grid-column-end: five;  
  grid-row-start: row1-start;  
  grid-row-end: 3;  
}
```



```
.item-b {  
  grid-column-start: 1;  
  grid-column-end: span col4-start;  
  grid-row-start: 2;  
  grid-row-end: span 2;  
}
```



Розташування елемента в сітці

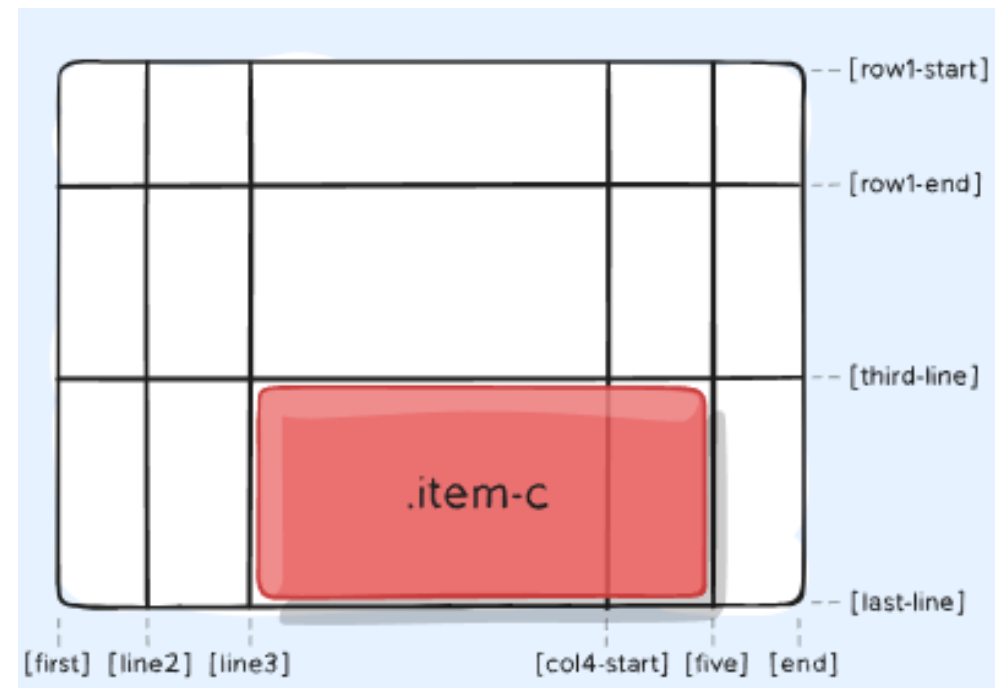
Властивості **grid-column** та **grid-row** являють собою скорочений запис для властивостей **grid-column-start** / **grid-column-end** та **grid-row-start** / **grid-row-end** відповідно.

Можливі значення:

<start-line> / **<end-line>** – кожна властивість може приймати ті самі значення, що й довга версія, включаючи **span**

Приклад:

```
.item-c {  
  grid-column: 3 / span 2;  
  grid-row: third-line / 4;  
}
```



Якщо значення **кінцевого рядка** не оголошено, елемент охоплюватиме **1 доріжку** за умовчанням.

Шаблон на основі назв областей (**grid-area**)

Властивість **grid-template-areas** визначає шаблон сітки, посилаючись на назви областей, які задані у властивості **grid-area**. Повторення назви області сітки призводить до того, що вміст охоплює ці комірки. Крапка означає порожню комірку. Сам синтаксис забезпечує візуалізацію структури сітки.

Доступні значення:

- <grid-area-name>** — назва зони сітки, зазначена с **grid-area**
- .** — крапка означає порожню комірку сітки
- none** — зони сітки не визначені

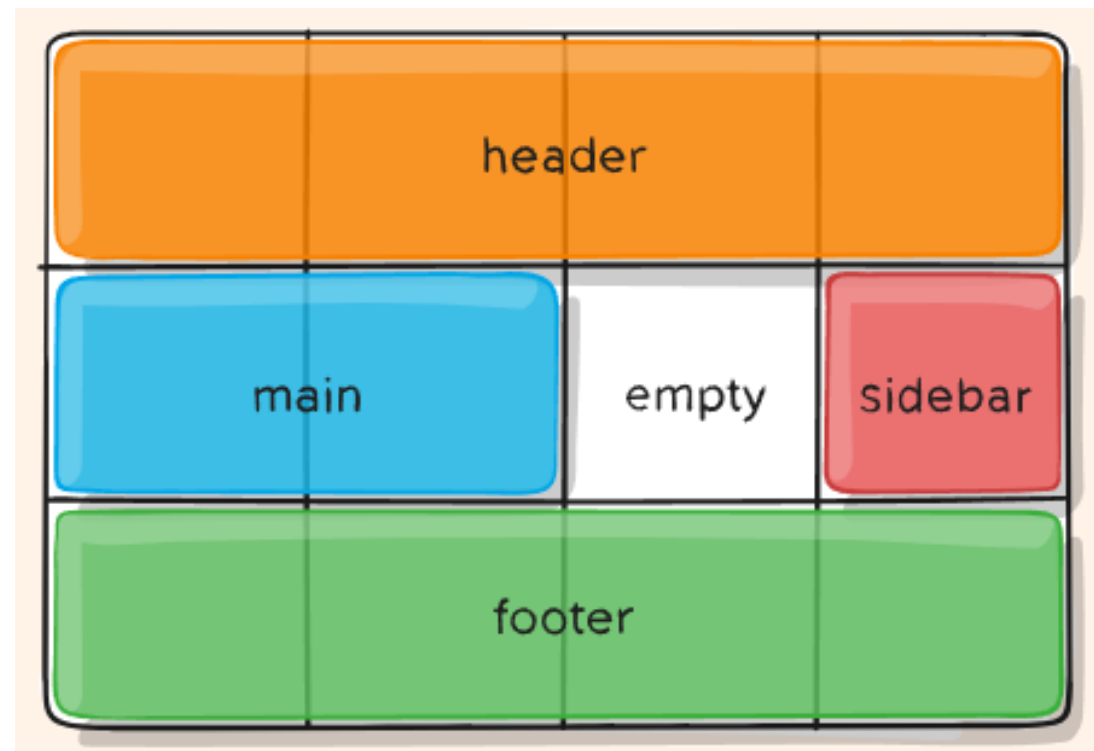
Синтаксис:

```
.container {  
  grid-template-areas:  
    "<grid-area-name> | . | none | ..."  
    "...";  
}
```

Шаблон на основі назв областей

Приклад створення сітки з чотирьох стовпців та трьох рядків. Весь верхній рядок буде складатися з області **header**. Середній рядок складатиметься з двох областей **main**, однієї порожньої клітинки (.) та однієї бічної панелі (область **sidebar**). Останній рядок — це весь нижній колонтитул (область **footer**).

```
.item-a { grid-area: header; }  
.item-b { grid-area: main; }  
.item-c { grid-area: sidebar; }  
.item-d { grid-area: footer; }  
  
.container {  
  display: grid;  
  grid-template-columns: 50px 50px 50px 50px;  
  grid-template-rows: auto;  
  grid-template-areas:  
    "header header header header"  
    "main main . sidebar"  
    "footer footer footer footer";  
}
```



Властивість **grid-template**

Властивість **grid-template** являє собою скорочену форму (скорочений спосіб) для встановлення властивостей **grid-template-rows**, **grid-template-columns** та **grid-template-areas** в одній декларації.

Така комплексна форма задання параметрів сітки **може ускладнювати розуміння коду**.

Може приймати значення:

none – встановлює всі три властивості на початкові значення

<grid-template-rows> / <grid-template-columns> – встановлює **grid-template-columns** і **grid-template-rows** в задані значення відповідно і встановлює **grid-template-areas** в **none**

```
.container {  
  grid-template: none | <grid-template-rows> / <grid-template-columns>;  
}
```

Приклад:

```
.container {  
  display: grid;  
  grid-template: repeat(4, 1fr) / repeat(3, 500px);  
}
```

Тобто, використовуючи **grid-template**, можна задати параметри всіх рядів та колонок сітки, розділяючи їх слешем /. **Спершу** йдуть **ряди**, а **потім колонки**, **не переплутайте!**

Властивість **grid-template**

У цій же властивості можна задавати й значення **grid-template-areas**, але тоді код ускладнюється і стає зовсім нечитабельним. Краще все ж таки використовувати цю властивість окремо:

```
.container {  
  display: grid;  
  grid-template:  
    [row1-start] «hdr hdr hdr " 25px [row1-end]  
    [row2-start] "ftr ftr ftr " 25px [row2-end]  
    / auto 50px auto;  
}
```



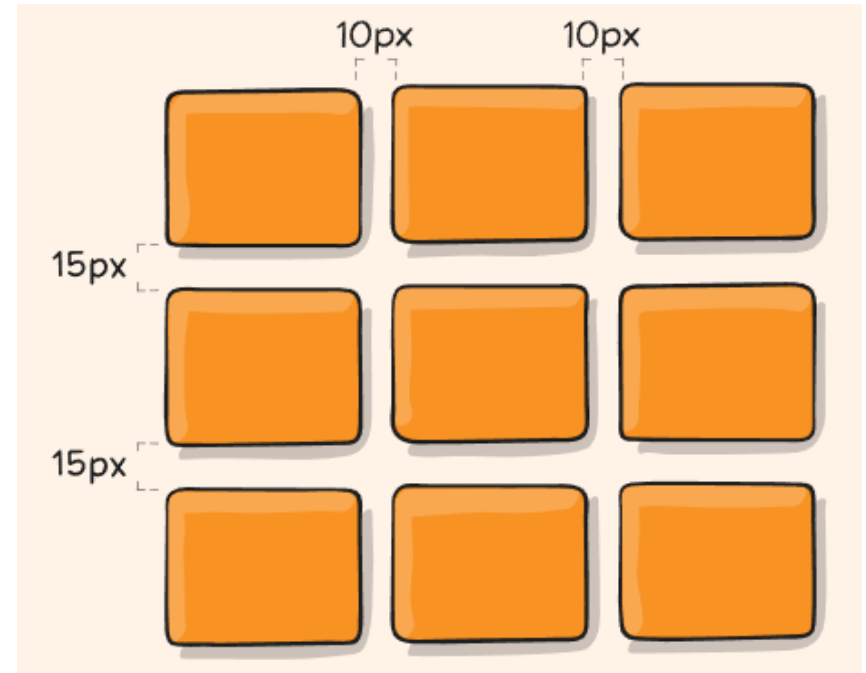
```
.container {  
  display: grid;  
  grid-template:  
    [row1-start] 25px [row1-end]  
    [row2-start] 25px [row2-end]  
    / auto 50px auto;  
  grid-template-areas:  
    "hdr hdr hdr "  
    "ftr ftr ftr ";  
}
```

Оскільки властивість **grid-template** не скидає **неявні** властивості сітки (**grid-auto-columns**, **grid-auto-rows**, та **grid-auto-flow**), що, мабуть потрібно робити в більшості випадків, рекомендується використовувати властивість **grid** замість **grid-template**.

Відступи між рядами/колонками сітки

Властивості **row-gap** та **column-gap** задають відступи між рядами та між колонками відповідно. Властивість **gap** – скорочений спосіб для запису значень властивостей **row-gap** та **column-gap** (значення розділяються пропуском).

```
.container {  
  display: grid;  
  grid-template-columns: 1fr 350px 1fr;  
  grid-template-rows: repeat(3, 150px);  
  /* Відступи між рядами */  
  row-gap: 15px;  
  /* Відступи між колонками */  
  column-gap: 10 px;  
  /* Відступи між рядими та колонками */  
  gap: 15px 10px;  
  /* Однакові відступи між рядими та колонками */  
  gap: 10px;  
}
```



В інспекторі відступи заштрихуються, то їх можна відрізнити від grid-елементів.

Вирівнювання елементів вздовж осі рядка

Властивість **justify-items** задає вирівнювання грід-елементів вздовж горизонтальною осі. Застосовується до всіх елементів усередині грід-контейнера

Можливі значення: `.container { justify-items: start | end | center | stretch; }`

start - вирівнює елемент по початковій (лівій) лінії;

end - вирівнює елемент по кінцевій (правій) лінії;

center - вирівнює елемент по центру грід-комірки;

stretch - розтягує елемент на всю ширину грід-комірки.

```
.container {  
  display: grid;  
  grid-template-columns: 1fr 400px 1fr;  
  grid-template-rows: repeat(3, 170px);  
  gap: 20px;
```

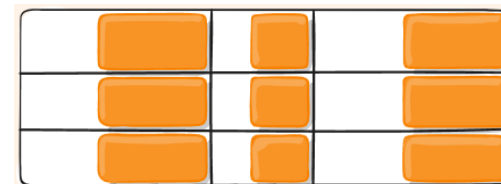
```
  justify-items: start;  
}
```

```
.item {  
  min-width: 300px;  
}
```

`.container { justify-items: start; }`



`.container { justify-items: end; }`



`.container { justify-items: center; }`



`.container { justify-items: stretch; }`



Вирівнювання елементів вздовж осі стовпця

Властивість **align-items** задає вирівнювання грід-елементів вздовж вертикальної осі. Застосовується до всіх елементів усередині грід-контейнера

Можливі значення: `.container {align-items: start | end | center | stretch | baseline ; }`

start - вирівнює елемент по початковій (лівій) лінії;

end - вирівнює елемент по кінцевій (правій) лінії;

center - вирівнює елемент по центру грід-комірки;

stretch - розтягує елемент на всю ширину грід-комірки;

baseline - вирівняти елементи вздовж базової лінії тексту.

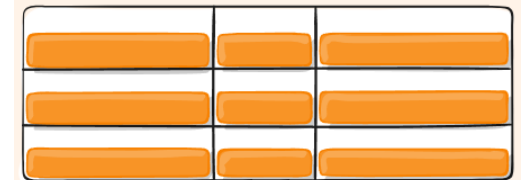
```
.container {  
  display: grid;  
  grid-template-columns: 1fr 400px 1fr;  
  grid-template-rows: repeat(3, 170px);  
  gap: 20px;  
  
  justify-items: start;  
}
```

```
.item { min-width: 300px; }
```

`.container {align-items: start; }`



`.container {align-items: end; }`



`.container {align-items: center; }`



`.container {align-items: stretch; }`



Вирівнювання елементів

Властивість **place-items** дозволяє у скороченій формі **одночасно** задати значення властивостей **align-items** та **justify-items**. Перше значення встановлює **align-items**, друге значення **justify-items**. Якщо друге значення не вказано, то перше значення присвоюється обом властивостям.

```
.container {  
  display: grid;  
  place-items: stretch end;  
}
```

Часто використовується для двонапрявленого центрування:

```
.center {  
  display: grid;  
  place-items: center;  
}
```

Вирівнювання сітки всередині контейнера

Іноді загальний розмір сітки може бути меншим за розмір її контейнера. Це може статися, якщо всі елементи сітки мають розмір негнучких одиниць (px). У цьому випадку можна встановити вирівнювання сітки всередині контейнера сітки. Властивість **justify-content** вирівнює сітку вздовж осі рядка. Тобто дана властивість **працює тільки тоді, коли загальна ширина стовпців менше ширини контейнера сітки** і потрібно розподілити вільний простір.

```
.container {  
  justify-content: start | end | center | stretch | space-around | space-between | space-evenly;  
}
```

Значення:

start – вирівнює сітку на одному рівні з початковим краєм контейнера

end – вирівнює сітку на одному рівні з кінцевим краєм контейнера

center – вирівнює сітку по центру контейнера

stretch – змінює розмір елементів сітки, щоб сітка заповнювала всю ширину контейнера

space-around – розміщує рівномірний простір між кожним елементом сітки з відступами половинного розміру на крайніх кінцях

space-between – розміщує рівномірний простір між кожним елементом сітки, без проміжків на крайніх кінцях

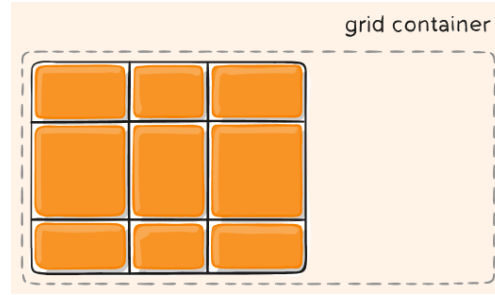
space-evenly – розміщує рівномірний простір між кожним елементом сітки, включаючи крайні кінці



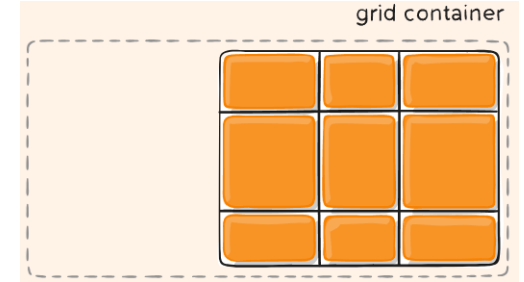
Вирівнювання сітки всередині контейнера

```
.container {  
  display: grid;  
  grid-template-columns: repeat(3, 300px);  
  gap: 20px;  
  justify-content: start;  
}
```

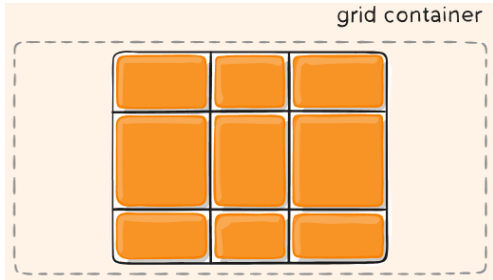
```
.container { justify-content: start; }
```



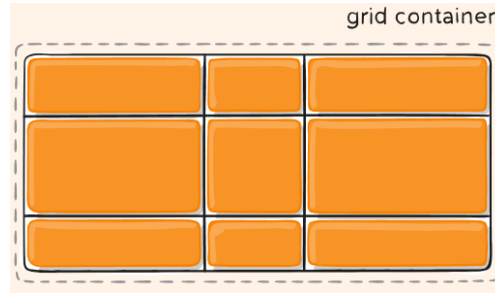
```
.container { justify-content: end; }
```



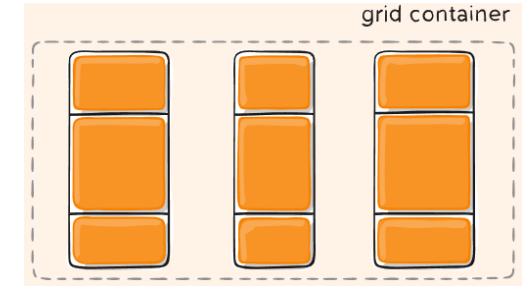
```
.container { justify-content: center; }
```



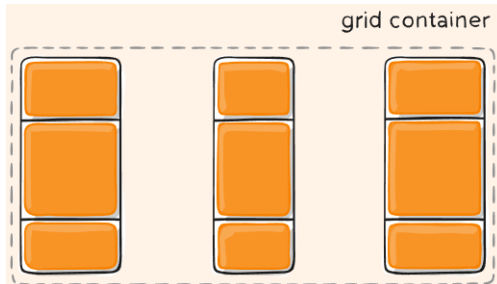
```
.container { justify-content: stretch; }
```



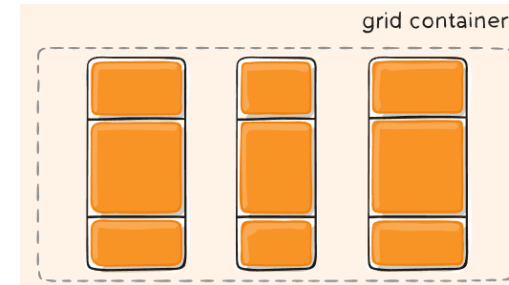
```
.container { justify-content: space-around; }
```



```
.container { justify-content: space-between; }
```



```
.container { justify-content: space-evenly; }
```

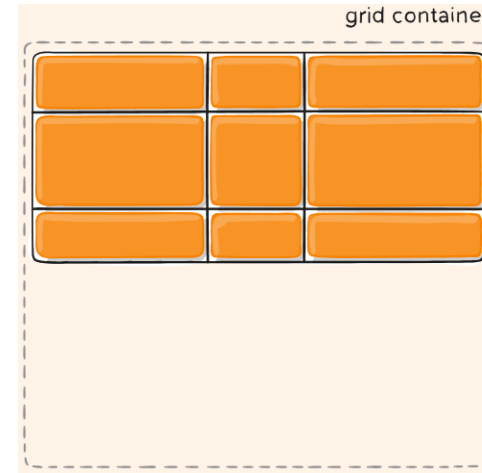


Вирівнювання сітки всередині контейнера

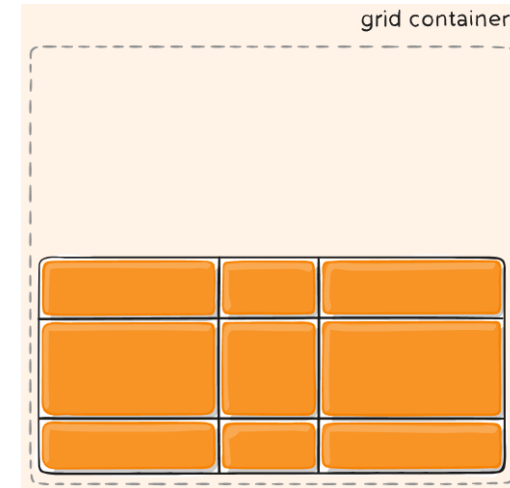
Властивість **align-content** вирівнює сітку вздовж осі стовпця (на відміну від **justify-content**, яка вирівнює сітку вздовж осі рядка). Тобто дана властивість **працює тільки тоді, коли загальна висота рядків менше висоти контейнера сітки** і потрібно розподілити вільний простір.

Можливі значення: **start** | **end** | **center** | **stretch** | **space-around** | **space-between** | **space-evenly**

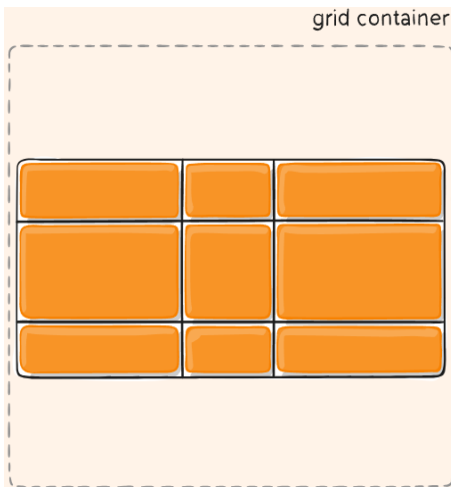
align-content: start;



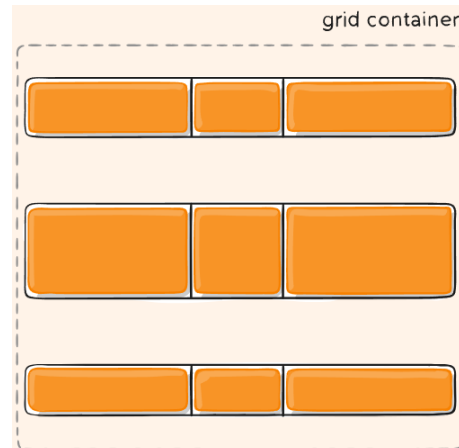
align-content: end;



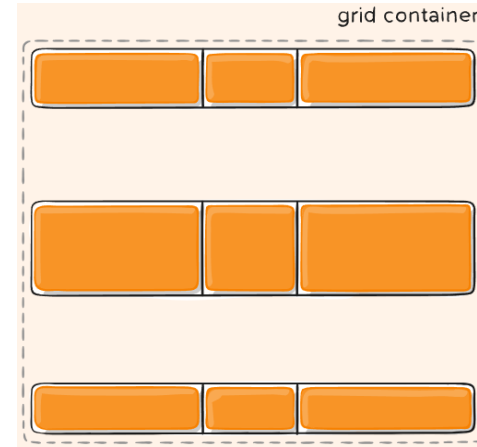
align-content: center;



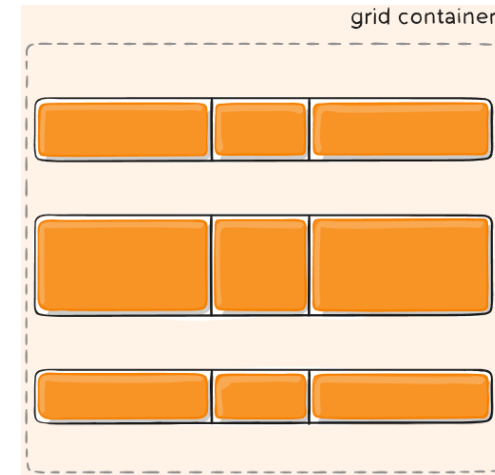
align-content: space-around;



align-content: space-between ;



align-content: space-evenly;



Вирівнювання сітки всередині контейнера

Властивість **place-content** дозволяє у скороченій формі **одночасно** задати значення властивостей **align-content** та **justify-content**. Перше значення встановлює align-content, друге значення justify-content. Якщо друге значення не вказано, то перше значення присвоюється обом властивостям.

```
.container {  
  display: grid;  
  place-content : stretch end;  
}
```

Може використовуватися для двонапрявленого центрування контенту:

```
.center {  
  display: grid;  
  place-content : center;  
}
```

Явна та неявна сітка

При створенні сітки за допомогою властивостей `grid-template-columns` та `grid-template-rows` задається так звана **явна сітка**. Визначена таким чином сітка має розміри, що дорівнює сумі всіх заданих треків.

Якщо ж **кількість grid-елементів перевищує** задану **кількість комірок**, або просто **елемент** розміщений так, що **виходить за межі створеної сітки**, це приводить до створення треків **неявної сітки**.

Задати **розмір неявних рядків** і **стовпчиків** можна за допомогою властивостей `grid-auto-rows` або `grid-auto-columns`. Ці властивості приймають список треків. Наприклад для того, щоб усі неявні колонки були висотою не менше **200px** і **збільшувалися**, якщо того вимає контент, можна вказати:

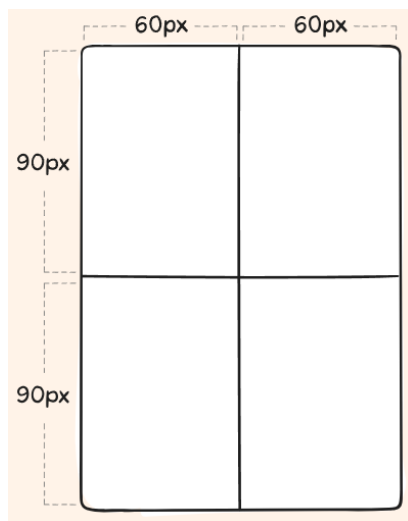
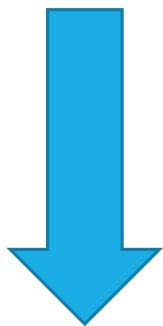
grid-auto-rows: `minmax(200px, auto)`

Якщо потрібно, щоб розмір першого неявного рядка визначався автоматично виходячи з вмісту, а другий мав фіксовану висоту 100px (і так далі, по черзі застосовуючи ці значення всіх наступних рядків), то властивості можна задати **кілька значень**.

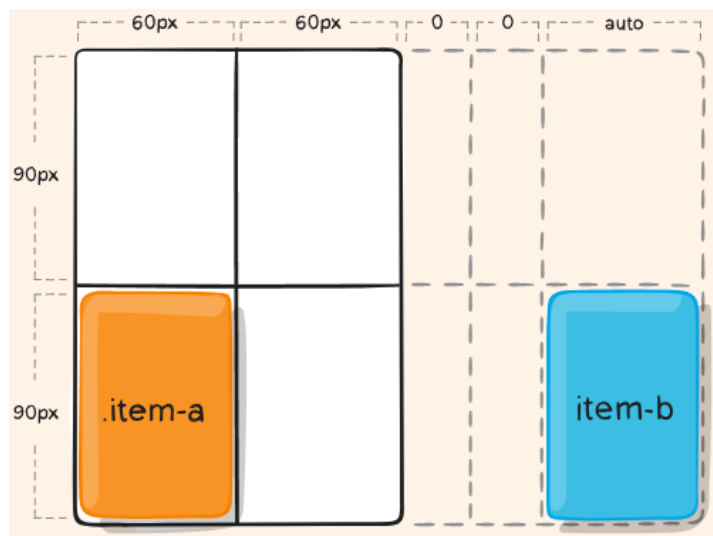
grid-auto-rows: `auto 100px`

Явна та неявна сітка

```
.container {  
  grid-template-columns: 60px 60px;  
  grid-template-rows: 90px 90px;  
}
```

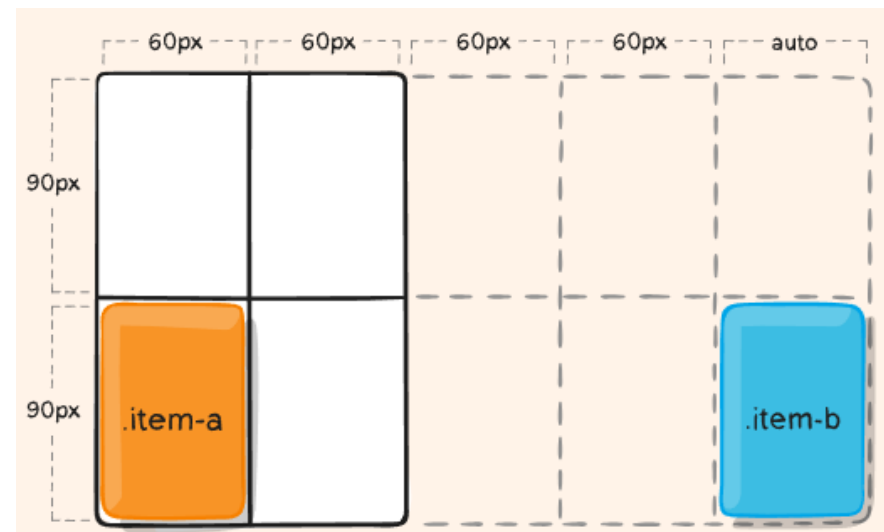


```
.item-a {  
  grid-column: 1 / 2;  
  grid-row: 2 / 3;  
}  
.item-b {  
  grid-column: 5 / 6;  
  grid-row: 2 / 3;  
}
```



Ми сказали `.item-b` починатися з лінії **5** і закінчуватися лінією **6**, але такі лінії (**5** та **6**) **не існують**. Тому для заповнення прогалів **створюються неявні доріжки** (треки) з шириною 0. Властивості використовувати **`grid-auto-columns`** і **`grid-auto-rows`** використовують для визначення ширини цих **неявних доріжок**:

```
.container {  
  grid-auto-columns: 60px;  
}
```



Алгоритм автоматичного розміщення

Якщо ґрід-елементів більше, ніж явно оголошених колонок чи рядів, вони автоматично розміщуються всередині батька. А ось яким чином - в ряд або колонку - можна вказати за допомогою властивості **grid-auto-flow**. За замовчуванням значення цієї властивості **row**, тобто зайві елементи будуть вибудовуватися в ряди в рамках явно заданих колонок.

Можливі значення:

row (значення за замовчуванням) — автоматично розміщені елементи вишиковуються в ряди;

column - автоматично розміщені елементи вибудовуються в колонки;

dense - браузер намагається заповнити дірки (порожні комірки) у розмітці, якщо дозволяють розміри елементів. Можна поєднувати з іншими значеннями.

Алгоритм автоматичного розміщення

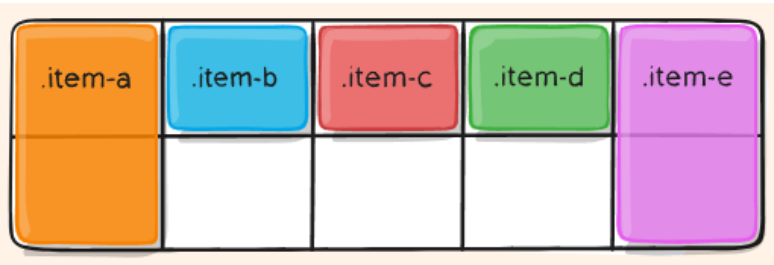
```
<section class="container">
  <div class="item-a">item-a</div>
  <div class="item-b">item-b</div>
  <div class="item-c">item-c</div>
  <div class="item-d">item-d</div>
  <div class="item-e">item-e</div>
</section>
```

```
.container {
  display: grid;
  grid-template-columns: repeat(5, 60px);
  grid-template-rows: repeat(2, 30px);
  grid-auto-flow: row;
}
```

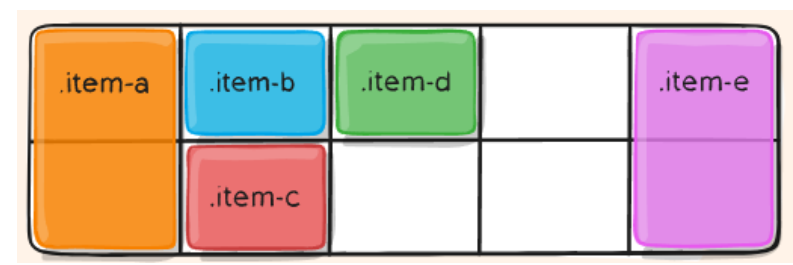
Розміщуючи елементи на сітці, вказуємо місця лише для двох із них:

```
.item-a {
  grid-column: 1;
  grid-row: 1 / 3;
}
.item-e {
  grid-column: 5;
  grid-row: 1 / 3;
}
```

Якщо встановити **grid-auto-flow** у значення **row**, то сітка виглядатиме так. Зверніть увагу, як три елементи, які ми не розмістили (item-b , item-c і item-d), розташовуються по доступних рядках:



Якщо встановити **grid-auto-flow** у значення **column**, то сітка виглядатиме так.



Властивість **grid**

Скорочена форма (shorthand), яка дозволяє задати значення **всьому** і **відразу**. А саме з допомогою властивості **grid** можна вказати значення для таких властивостей:

`grid-template-rows;`
`grid-template-columns;`
`grid-template-areas;`
`grid-auto-rows;`
`grid-auto-columns;`
`grid-auto-flow.`

Перед тим як спокуситися можливістю розписати все в одній властивості, потрібно двічі (або навіть тричі) подумати про читабельність коду.

Можливі значення:

none - значення за замовчуванням. Це ключове слово скидає значення всіх властивостей, що входять в цей shorthand.

```
.container {  
  display: grid;  
  grid: none;  
}
```

Властивість **grid**

Можна вказати допустимі значення для шорткату grid-template:

```
.container {  
  display: grid;  
  grid: repeat(4, 150px) / 1fr 200px 1fr;  
}
```

У тому числі можна вказати **імена ліній**:

```
.container {  
  display: grid;  
  grid:  
    [row1-start] 25px [row1-end row2-start] 25px [row2-end]  
    / auto 50px auto;  
}
```

Можна встановити розміри **колонок** і **рядів**. Створимо два ряди та дві колонки:

```
.container {  
  display: grid;  
  grid: 200px 100px / 30% 30%;  
}
```

Властивість **grid**

auto-flow — це ключове слово, яке дає браузеру зрозуміти, що створювати за необхідності: колонки чи ряди.

Якщо **auto-flow** стоїть праворуч від слеша, то створюватимуться автоматичні колонки:

```
.container {  
  display: grid;  
  grid: 200px 100px / auto-flow 30%;  
}
```

Якщо **auto-flow** стоїть ліворуч від слеша, то створюватимуться автоматичні ряди:

```
.container {  
  display: grid;  
  grid: auto-flow 30% / 200px 100px;  
}
```

dense - вказує браузеру, як елементи повинні намагатися заповнити вільні комірки.

Важливо писати це слово відразу після **auto-flow**

```
.container {  
  display: grid;  
  grid: auto-flow dense 30% / 200px 100px;  
}
```

Властивість **grid**

Скорочена форма запису

Повна форма запису

Задаємо розміри та кількість колонок і рядів:

```
.container {  
  grid: 50px 150px / 2fr 1fr;  
}
```



```
.container {  
  grid-template-rows: 50px 150px;  
  grid-template-columns: 2fr 1fr;  
}
```

Задаємо розміри та кількість рядів, а також автоматичні колонки:

```
.container {  
  grid: 200px 1fr / auto-flow 200px;  
}
```



```
.container {  
  grid-auto-flow: column;  
  grid-template-rows: 200px 1fr;  
  grid-auto-columns: 200px;  
}
```

Задаємо параметри сітки з іменами ліній та іменами областей

```
.container {  
  grid:  
    [row1-start] "hd hd hd" 1fr [row1-end]  
    [row2-start] "ft ft ft" 25px [row2-end]  
    / auto 50px auto;  
}
```



```
.container {  
  grid-template-areas:  
    "hd hd hd"  
    "ft ft ft";  
  grid-template-rows: [row1-start] 1fr [row1-end row2-start] 25px [row2-end];  
  grid-template-columns: auto 50px auto;  
}
```

Вирівнювання окремих комірок (*-self)

За допомогою властивості **justify-self** можна встановити горизонтальне вирівнювання для окремого елемента, відмінне від вирівнювання, заданого грід-батьку. Значення застосовується до окремого елемента сітки (певної окремої комірки сітки).

Можливі значення:

start – вирівнює елемент сітки на одному рівні з початковим краєм комірки;

end – вирівнює елемент сітки на одному рівні з кінцевим краєм комірки;

center – вирівнює елемент сітки по центру комірки;

stretch – заповнює всю ширину комірки (за умовчанням).

Властивість **align-self** вирівнює окремий елемент по вертикальній осі. Можливі значення аналогічні значенням властивості **align-self**

Властивість **place-items** - скорочений форма для вказання значень відразу для **align-items** та для **justify-items**. Вказувати потрібно саме так: **<align-self>** / **<justify-self>**.

Grid vs browsers DevTools

Поекспериментувати з параметрами grid-контейнера можна за допомогою засобів розробника (тобто змінити, застосувати і переглянути результат змін одночасно).



Grid vs browsers DevTools

Відобразити та переглянути налаштування сітки можна у закладці Layout

Popular Activities

1 2 3 4 5 6

1 im1 2 im2 3 im3 4 im4 5 im5 6

Tennis Running Cycling

2 im5 3 im6

Yoga Hiking Swimming

3 -6 -5 -4 -3 -2 -1

```
</div>
</div>
</body>
```

html body div.activities div.activity-container

Styles Computed **Layout** Event Listeners

▼ Grid

Overlay display settings

Show line numbers ▼

☐ Show track sizes

☒ Show area names

☒ Extend grid lines

Grid overlays

☒ div.activity-container

Підсумок

Навчилися використовувати технологію верстки grid, тобто :

Вивчили основні терміни.

Навчилися встановлювати параметри контейнера та розташовувати елементи в сітці

Навчилися використовувати шаблон на основі назв областей (grid-area)

Навчилися налаштовувати відступи

Навчилися вирівнювати елементи

Навчилися вирівнювати сітку всередині контейнера

Ознайомилися з комплексною властивістю grid

Ознайомилися Grid vs browsers DevTools

Попрактикуйтеся на

<https://cssgridgarden.com/>